

Eurskem AI

10-Minute Technical Walkthrough

The spoken script, the screen to be on for every beat, the code to open, and the exact artefact that backs each claim — with every screenshot captured from the running application.

Purpose	Portfolio or technical interview video
Target length	About 1,300 spoken words · 10 minutes at 125–135 wpm
Style	Calm, clear, business-first, then technical depth
Demo workflow	Concept Note to 10-Page Methodology Section (14 nodes, 18 edges)
Demo run	ed462b3e — completed, 23 min 33 s, \$6.6495
Failure demo	3d46a8ee — failed at graphnormalizer_1
Evidence demo	1f2c3a4d — evidence stage, 19 claims, 0 verified
Code snapshot	main at 7ca16b2
Document date	9 August 2026

How to use this document

Each beat gives you three things in a fixed order: **On screen** (what to have open before you start talking), **Say** (the words, verbatim), and the supporting screenshot or code. Amber presenter notes flag places where the original script promised something the current build does not surface, along with what to show instead. Rehearse from the Say blocks alone; use the rest to set up.

Prepare the run | open the tabs | then speak

Before you record

Every claim in the script has a specific artefact behind it. Have them open.

Services and tabs

Tab	Route	Used in beat
Workflow Library	<code>/library</code>	1, 2
Guided Run	<code>/history</code> → run <code>ed462b3e</code> → Open in Guided	3
Cockpit (completed)	same run → Open in Cockpit	3
Cockpit (failed)	<code>/history</code> → run <code>3d46a8ee</code> → Open in Cockpit	3, 8
Builder	<code>/builder/concept_note_to_10-page_methodology_section1</code>	4
Editor	<code>app/runtime/compiler.py</code> at <code>_make_runtime_fn</code> (line 63)	5
Evidence record	run <code>1f2c3a4d</code> → <code>verify_evidence.qa_report</code>	6
Cost ledger	Mongo <code>cost_ledger</code> , filtered to <code>ed462b3e</code>	7

Bring up MongoDB, Weaviate, MinIO and Redis from `docker-compose.yml`, start the FastAPI backend and the Vite dev server, and confirm `/health` reports `ready: true` before recording — a readiness failure mid-take is the one interruption that cannot be edited out.

Three corrections to the original script, resolved here

- 1. Per-node cost is not in the UI.** The Cockpit node inspector says so itself: "Token/cost breakdown isn't joined into this view yet." Beat 7 therefore reads the ledger record directly and says so out loud, rather than implying a panel that does not exist.
- 2. A fallback record does exist.** The original script said to show one only if it is real. It is: `outline` requested `claude-sonnet-4-5` and `gpt-5.6-terra` ran. Beat 7 uses it.
- 3. The demo run has no verified claims to trace.** Its evidence base is bounded Deep Research dossiers; Cockpit shows `Sources (0)`. Beat 6 therefore traces the evidence lifecycle in a run that genuinely exercised it — and the honest result there is zero verified claims, which is a stronger demonstration than a happy path.

Pacing

Rehearse at 125–135 words per minute and pause when changing screens. The Say blocks total about 1,300 words. Claim nothing about adoption, accuracy, savings or production health — the parts that are evidenced are more persuasive without it.

0:00-0:35 Introduction

~85 words

ON SCREEN

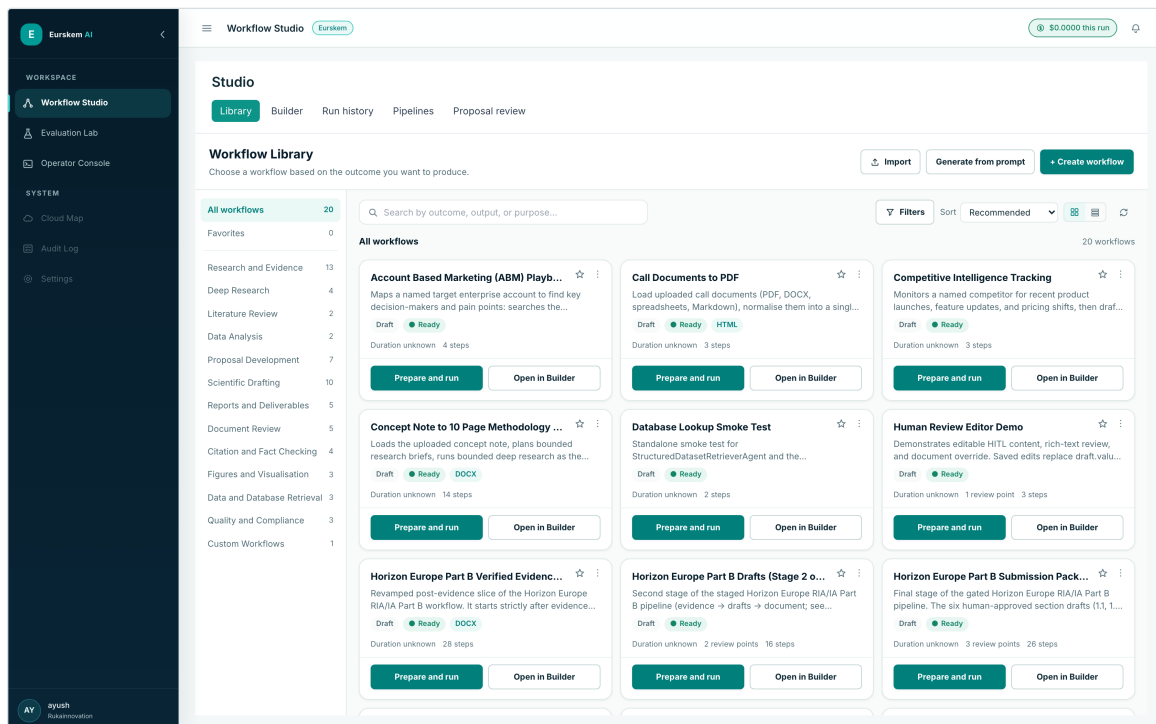
Workflow Library at `/library`, with the concept-note workflow visible in the grid. Sidebar expanded. Cost badge reading `$0.0000 this run`.

SAY

"Hello, I am Ayush Khandelwal. This is Eurskem AI, a reusable platform for building and operating evidence-grounded AI workflows.

I will demonstrate it through proposal development, where research, evidence checking, drafting, review, and cost control must work together. A chatbot hides these steps. This platform makes them visible, configurable, testable, and recoverable.

I will begin with the non-technical user experience and then move into the Builder, runtime, research integrity, and cost controls."



Shot 1 • `/library` — twenty workflows discovered from the `workflows/` directory, grouped by outcome rather than by graph shape. Let the category counts sit on screen for a beat: they are the registry's own capability declarations, and they make "reusable platform" visible before you have to assert it.

0:35–1:20

Workflow Library and business goal

~120 words

ON SCREEN

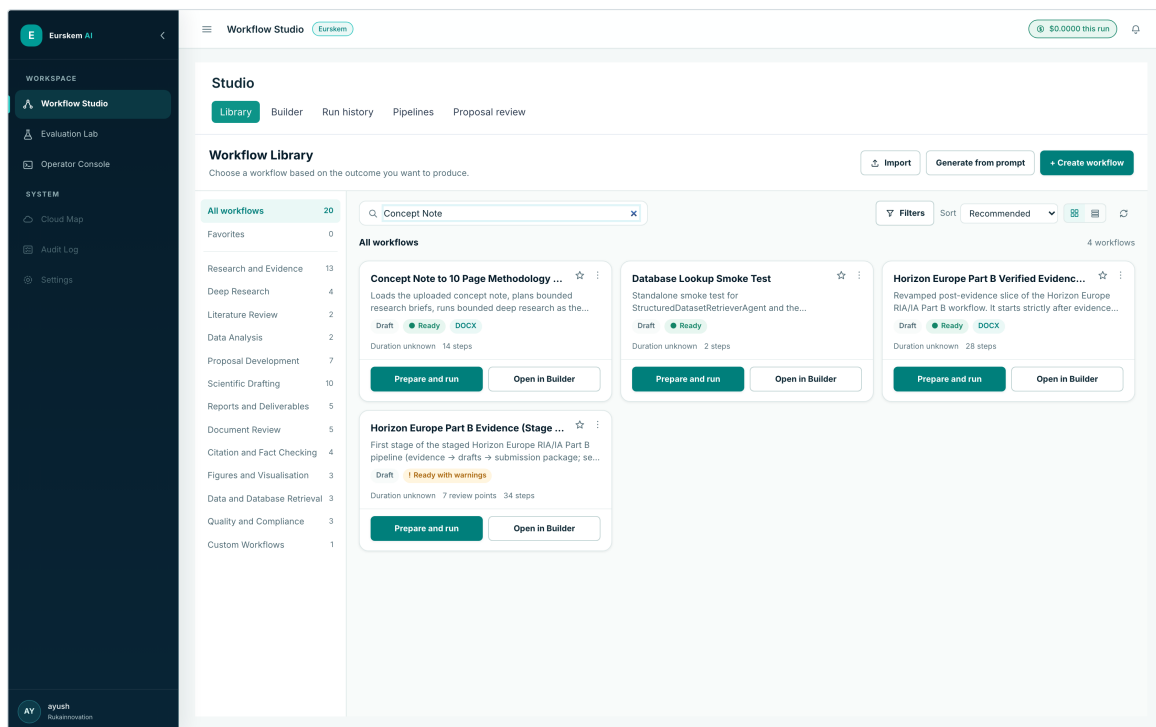
Type "Concept Note" into the library search so the flagship card is isolated. Point at the description, the **14 steps** badge and the **DOCX** output tag.

SAY

"I start in the Workflow Library. Workflows are presented by their business goal, not only by a technical graph.

This workflow takes a call or request for proposal, gathers relevant evidence, develops structured content, introduces human review points, and produces a cited deliverable. Its definition is versioned, while the underlying node types and runtime are reusable. Proposal generation is therefore one use case of the platform; it is not hard-coded as the platform itself.

For this recording, I am using prepared data and an existing run so the demonstration remains repeatable and does not depend on live provider response times."



Shot 2 · library filtered to the flagship — the card leads with the outcome, the step count and the output format. The neighbouring cards (Horizon Europe Part B Verified Evidence, Database Lookup Smoke Test) are worth a half-second of screen time: they are the same runtime, and they are what "platform, not product" means in practice.

If asked "is this workflow special-cased anywhere?"

Open `workflows/concept_note_to_10-page_methodology_section1.yaml`. It is 352 lines of the same `WorkflowSpec` contract every other workflow uses — typed inputs, nodes, edges, and Guided Run copy. There is no proposal-specific branch in the runtime.

1:20–2:25 Guided Run, and the same run in Cockpit

~185 words

ON SCREEN

`/history` → run `ed462b3e` → **Open in Guided**. Show the stage rail, the attention panel and the outputs. Then **Open in Cockpit** from the same run, and switch to the failed run `3d46a8ee` to show live node states.

SAY

"Guided Run is designed for project administrators and domain experts. Instead of node IDs and raw payloads, it shows business stages: understanding the request, collecting evidence, developing content, reviewing, and producing the output.

For the current stage, the user can see what is happening, why it matters, and what will receive its result. Approvals and failures appear in one attention area, so the user does not need to inspect a technical graph.

I will now open the same run in Cockpit. This does not start another execution. The run ID, workflow version, outputs, and audit context remain the same.

Cockpit answers a more technical question: which node is running, paused, completed, or failed? Live events update the graph, while a human-review node can pause execution for approval, rejection with a reason, or an edited response. Guided Run and Cockpit therefore support different users without creating two workflow engines."

The screenshot displays the Eurskem AI Studio interface. On the left is a sidebar with navigation options: Workspace, Workflow Studio (selected), Evaluation Lab, Operator Console, System, Cloud Map, Audit Log, and Settings. The main area is titled 'Studio' and shows a workflow titled 'Concept Note to 10-Page Methodology Section'. The workflow status is 'COMPLETED' with '1 item need review'. It shows '5 of 5 stages complete' and '23 min 33 sec elapsed'. The current stage is 'Finalise'. Below this, a stage rail shows five stages: Prepare, Understand, Gather, Create, and Finalise, all marked as 'Completed'. The 'Render docx' task is highlighted as 'CURRENT WORK'. An 'ATTENTION' panel indicates that 'Render docx' review is required. The 'LATEST RESULT' section shows 'DOCX deliverable' and 'HTML deliverable' ready for download. The 'COMPLETED WORK' section shows 'Contributions to the final result' including 'Graphnormalizer 1'.

Shot 3 · Guided Run, run `ed462b3e` — five stages complete, 23 min 33 s elapsed, a plain-language account of the current work and why it matters, and a single "What needs you" panel holding the one item that requires a decision. Nothing in the primary reading path is a node ID, a prompt or a payload.

The screenshot displays the Eurskem AI Workflow Studio interface. On the left is a dark sidebar with navigation options: Workspace (Workflow Studio, Evaluation Lab, Operator Console) and System (Cloud Map, Audit Log, Settings). The main area is titled 'Studio' and includes tabs for Library, Builder, Run history, Pipelines, and Proposal review. A top bar shows a balance of \$0.0000 and a 'this run' button. The workflow 'Concept Note to 10-Page Methodol...' is shown as 'Failed' with a start time of 9:29:27 AM and 4m 6s elapsed. A table shows execution statistics: Running (0), Completed (2), Waiting (0), Paused (0), Failed (1), Skipped (11), and Cancelled (0). A search bar for nodes is present, listing 14 nodes including 'load_files', 'concept_frame', 'planner', 'deep_research', 'outline', and several 'draft_chunk' nodes. The central canvas shows a flow graph with nodes and connections. On the right, a red error message states: 'Workflow failed: OpenAIGateway: empty structured response for GraphExtraction. refusal=None'. Below this is a green button labeled 'Open Run History and retry'.

Shot 4 · Cockpit, run 3d46a8ee — the same workflow rendered as execution state: lifecycle counters, a per-node status list, the live graph, and the failure that stopped this run with a retry route beside it.

Presenter note — which run to use for the Cockpit graph

Cockpit switches to the **output viewer** when a run's status is `completed` (`ui/src/modes/studio/Cockpit.tsx:370`). Opening `ed462b3e` lands on the rendered DOCX and its Variables / Sources / Audit / Score tabs — good for beat 6, wrong for showing node lifecycle. Use the failed run `3d46a8ee` for the graph. Say the quiet part out loud: "this is an earlier attempt that failed, which is the honest way to show what the graph does when something goes wrong." It costs four seconds and buys credibility.

2:25–3:55 Builder controls

~240 words

ON SCREEN

`/builder/concept_note_to_10-page_methodology_section1`. Show the four areas, then select `deep_research` and step the inspector through **Configure** → **Map data** → **Test** → **Checks**, then **Versions**.

SAY

"The Builder has four stable areas: the action bar, a searchable and categorised node library, the canvas, and a persistent Inspector.

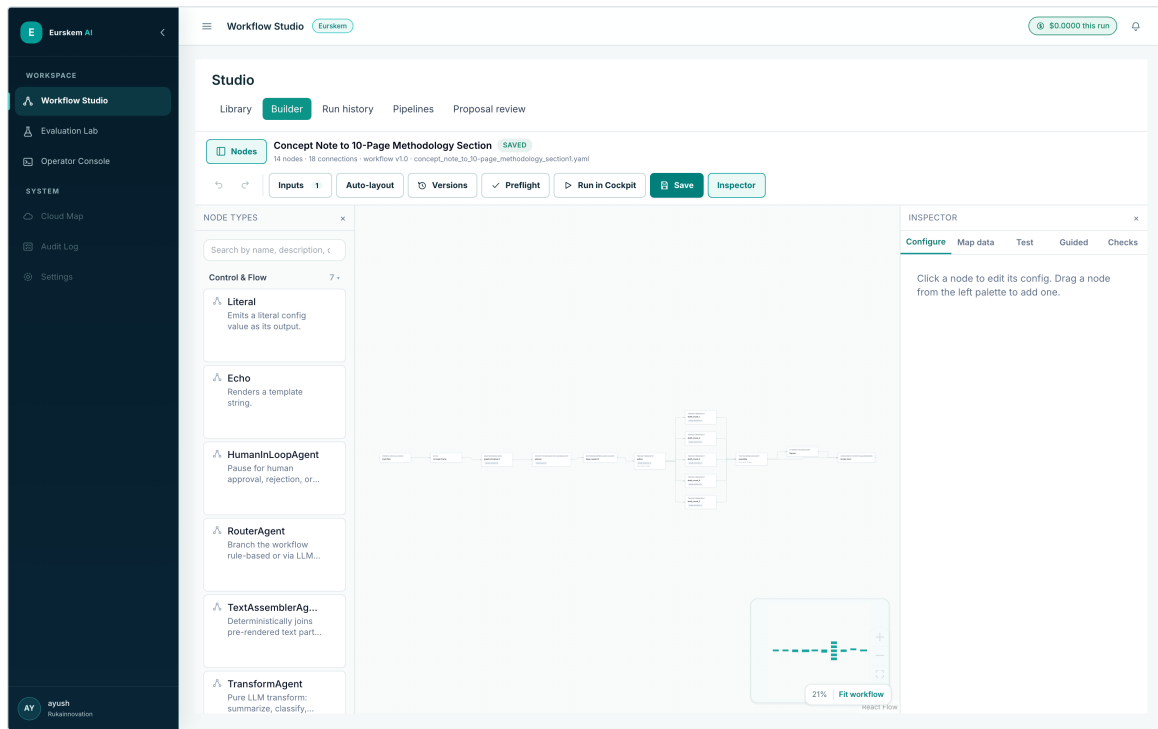
The library is populated from the backend node registry. Each capability declares its configuration, inputs, and outputs. Selecting a node creates a typed form from that contract, which reduces invalid free-text configuration.

In Map Data, I can connect an input to an upstream output. The variable picker only exposes data available at this point in the graph, and the saved reference remains explicit.

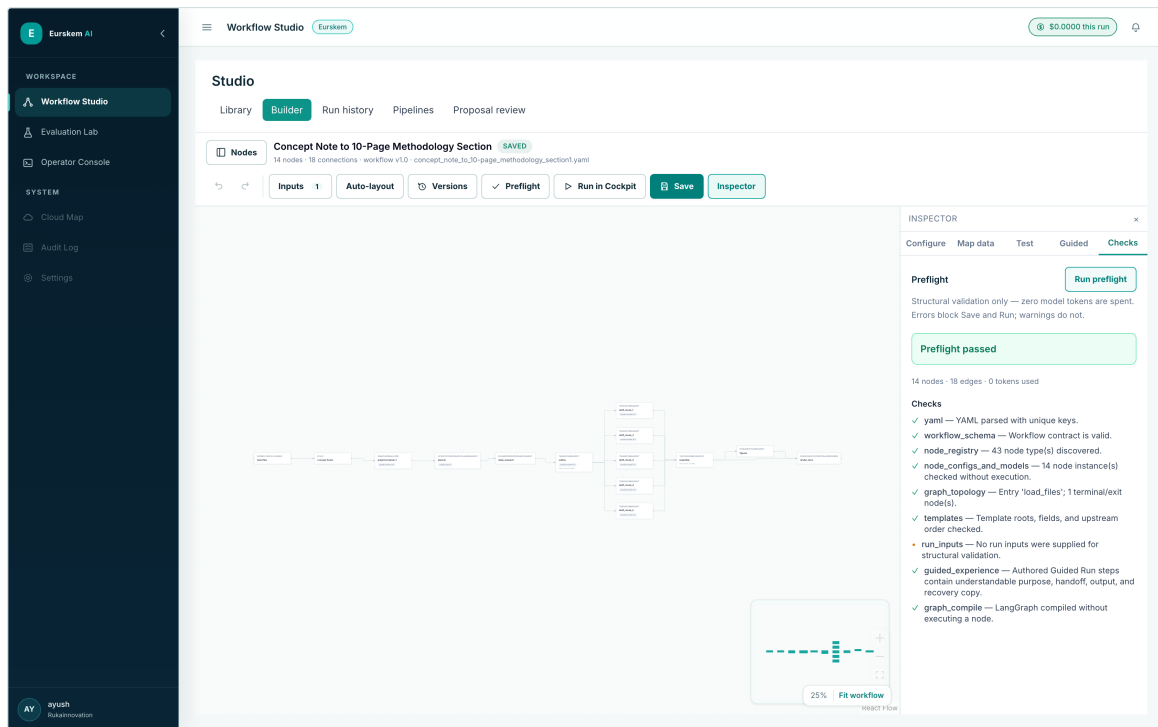
Before saving or running, preflight validates the workflow without spending model tokens. It catches missing configuration, unsafe references, orphaned edges, invalid routes, or incomplete user guidance. Each issue links back to the affected node.

I can also prepare a node-level or branch-level test instead of executing the complete workflow. It still uses the normal preflight and runtime path.

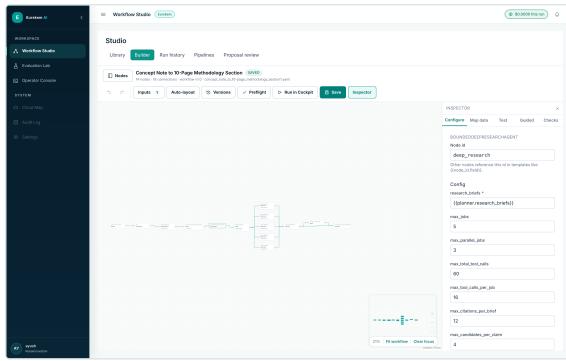
Incomplete changes are autosaved as a recoverable draft, separately from the last executable workflow. A strict save creates an immutable version. Version history lets me compare, restore, or recover work without allowing an incomplete draft to replace a known runnable version."



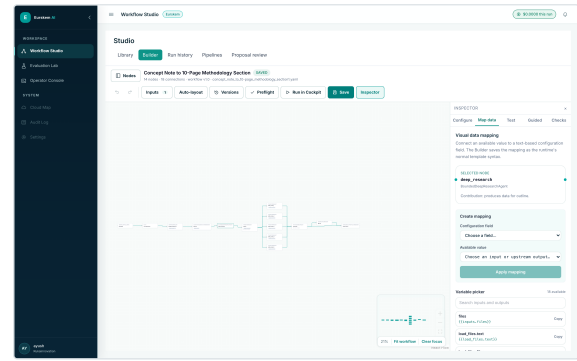
Shot 5 · the four areas — action bar, registry-driven node library, canvas, persistent inspector. Say the counts as you scroll the library: Control & Flow 7, Research & Discovery 5, Evidence & Retrieval 9, Proposal Engineering 9, Multimodal 4, Document Rendering & Export 7, Integrations 2 — 43 node types, all discovered, none hand-listed.



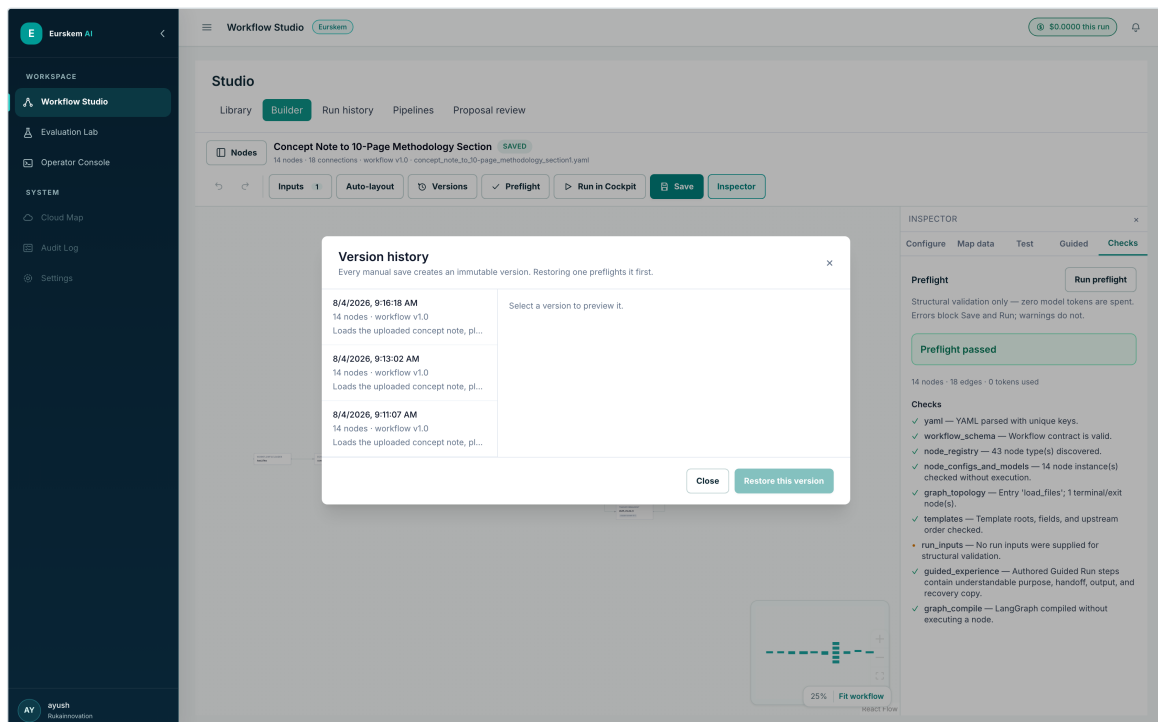
Shot 6 · the line that carries this beat — "Preflight passed · 14 nodes · 18 edges · 0 tokens used." Pause on it. Nine checks ran, including a real LangGraph compile, and the token counter is a product surface rather than an implementation detail. Point at `run_inputs`, the one neutral dot: no run inputs were supplied for a structural check, and it is deliberately not an error.



Shot 7 · typed configuration — the research node's bounds are declared numeric fields, not prose in a prompt.



Shot 8 · explicit mapping — the picker offers only what is reachable here and writes the reference preflight will check.



Shot 9 · immutable versions — three recorded versions of this workflow. Restoring one runs preflight first; the autosaved draft lives elsewhere, so an incomplete edit can never overwrite a runnable definition.

If asked how the save is made safe app/workflow/builder_store.py:34-144 @ 7ca16b2

```
def _atomic_write(path: Path, text: str) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    temporary = path.with_name(f".{path.name}.{uuid.uuid4().hex}.tmp")
    temporary.write_text(text, encoding="utf-8")
    temporary.replace(path)          # atomic – no half-written workflow

def save_workflow(self, name: str, yaml_text: str) -> str:
    path = self.workflow_path(name)
    if path.exists():
        current = path.read_text(encoding="utf-8")
        if current != yaml_text:
            self.record_version(name, current)    # keep what is being replaced
    _atomic_write(path, yaml_text)
    version_id = self.record_version(name, yaml_text)
    self.delete_draft(name)
    return version_id
```

3:55–5:25 Trace a node through `_make_runtime_fn`

~230 words

ON SCREEN

Editor at `app/runtime/compiler.py`, line 63. Scroll slowly through the `reuse` branch, the `with_context` binding, the pause check, and `output_schema(**output)`. Do not read the code aloud — narrate what each region guarantees.

SAY

"The most important runtime boundary is `_make_runtime_fn` in the compiler.

The validated workflow is compiled into a LangGraph state graph, and every node is added through this wrapper rather than being executed directly. This gives all node types the same operational controls.

The wrapper receives shared typed state containing the run ID, previous outputs, citations, and audit information. Large files are represented by object-store keys rather than raw bytes, keeping the state smaller and serialisable.

The LLM gateway is then bound to the current run and node. Any model call made through it can be attributed correctly. Node lifecycle events — started, completed, paused, or failed — are published to the event layer and update Cockpit.

Typed configuration and structured outputs provide validation boundaries. If execution fails, the error stays connected to the node and run. The executor and checkpointers retain the state needed for an allowed retry, resume, or human decision.

This single boundary keeps context, events, audit, validation, cost attribution, and recovery consistent. Implementing these separately inside each node would create drift as the library grew."

Code · the one node boundary, abridged app/runtime/compiler.py:63-260 @ 7ca16b2

```
def _make_runtime_fn(instance, bus: RunEventBus | None, services: dict):
    node_id = instance.node_id
    type_name = instance.type_name

    async def runtime_fn(state: dict) -> dict:
        run_id = state.get("inputs", {}).get("SYSTEM.run_id")
        session_id = state.get("session_id", "unknown")
        audit = services.get("audit_db")

        # (1) RETRY REUSE — returns before any gateway is bound,
        #     so a replayed LLM node consumes zero new tokens.
        cached_result = (services.get("reused_node_results") or {}).get(node_id)
        if cached_result is not None:
            ... # publish node_reused, write audit, re-checkpoint, return

        # (2) COST ATTRIBUTION — with_context() returns a shallow clone,
        #     so parallel branches get isolated context without locking.
        llm = services.get("llm")
        node_services = {**services, "llm": llm.with_context(
            run_id=run_id or "unknown", session_id=session_id, node_id=node_id,
            ledger=services.get("cost_ledger"), event_bus=bus, node_type=type_name,
            allowed_models=getattr(instance, "_allowed_models", None),
            routing_policy=getattr(instance, "_model_routing", None),
            entity_tokenizer=services.get("entity_tokenizer"),
            collection_id=state.get("collection_id", "default"),
            processing_mode=(getattr(instance, "_data_protection_mode", None)
                             or services.get("entity_protection_mode")),
        )}
        instance.services = node_services

        # (3) EVENTS + AUDIT — one publish/write pair for every node type.
        await bus.publish(RunEvent(type="node_started", run_id=run_id, ...))
        await write_audit_event(audit, run_id, session_id, node_id, NODE_START)

        with metrics.track_node(type_name):
            # (4) TEMPLATE RESOLUTION against live state.
            resolved = resolve(instance.config.model_dump(), state)
            await record_node_started(audit, ...,
                                     node_input=build_node_input(state, resolved))

            # (5) COOPERATIVE PAUSE — checked at the node boundary, because
            #     nothing can interrupt an in-flight provider call.
            if await is_pause_requested(audit, run_id=run_id, session_id=session_id):
                interrupt({"kind": "user_requested_pause", "node_id": node_id})

            # (6) INVOKE + VALIDATE.
            output = await instance.run(state, resolved)
            validated = instance.output_schema(**output)
```

The sentence to land this beat

"Three of these are only correct because they live in one place. Retry reuse returns before a gateway exists, so a replayed node cannot spend money. Cost context is cloned per call, so five parallel drafting branches attribute correctly without a lock. And pause is checked at the node boundary, because you cannot interrupt an in-flight provider call. Written per node type, those would drift across forty-three node types — and drift here is silent."

5:25–7:05 Trace a claim to exact evidence — and to a refusal

~260 words

ON SCREEN

The evidence-stage run `1f2c3a4d`: its `acquire_research_sources.report`, then `verify_evidence.qa_report`, then a single `verified_claims` entry. Have `app/evidence/models.py` open in a second tab for `EvidencePolicy`.

SAY

"Next, I will trace one research claim. The central rule is that discovering a source is not the same as verifying evidence.

Candidate discovery searches approved scholarly and other sources and records identifiers and metadata. These are candidates only. A matching title or abstract does not automatically become a proposal citation.

The following stages obtain available content, remove duplicates, and rank passages for the research question. Exact-passage verification then records the source, the supporting passage, its location, and the claim it is intended to support.

I want to show you a run where that gate actually held. This evidence stage examined nineteen critical claims. Source acquisition resolved no canonical URLs into immutable full text, so verification had nothing to quote against. The result is nineteen unresolved claims, nineteen blocking issues, and zero verified claims. The acquisition node records it in one line: no search snippet and no Deep Research prose was promoted to verified evidence.

That is the behaviour I designed for. A system optimised to look productive would have cited the dossier prose it already had, and nineteen plausible sentences with plausible references are indistinguishable from good work until an evaluator checks them.

There is an important distinction: retrieval says a passage is relevant; exact-passage verification says it supports the stated claim. Neither alone proves that the source is authoritative. Important decisions still require source-quality rules and domain review."

Artefact 1 · the acquisition node's own report `run 1f2c3a4d · acquire_research_sources.report`

```
"Acquired 0 immutable cited source version(s) directly from canonical URLs;
rejected 0. No search snippet or Deep Research prose was promoted to
verified evidence."
```

Artefact 2 · the QA report that refuses to certify the run `run 1f2c3a4d · verify_evidence.qa_report`

```
{
  "policy_version": "eurskem-evidence-v2.0",
  "claims_examined": 19,   "critical_claims": 19,
  "verified_claims": 0,    "qualified_claims": 0,
  "unresolved_claims": 19,
  "sources_retained": 0,   "sources_rejected": 0,
  "full_text_evidence_rate": 0.0,
  "exact_locator_rate": 0.0,
  "critical_independent_support_rate": 0.0,
  "warnings": ["No proposal-grade citations passed all hard gates."]
}
```

Artefact 3 · one claim, refusing to be cited `run 1f2c3a4d · verify_evidence.verified_claims[0]`

```
{
  "claim_id": "CL-1",
  "materiality": "critical",
  "evidence_requirement": "2 independent full-text source(s), exact locator,
                           contradiction search",
  "final_status": "not_found",
  "verified_sentence": "",
  "source_ids": [], "evidence_link_ids": [],
  "unresolved_issue": "0/2 independent supporting sources;
                       contradiction search not completed",
  "human_review_required": true
}
```

Why it refused — the defaults are fail-closed `app/evidence/models.py:46-70 @ 7ca16b2`

```
class EvidencePolicy(BaseModel):
    model_config = ConfigDict(extra="forbid")    # a typo fails loudly

    policy_version: str = "eurskem-evidence-v2.0"
    minimum_independent_sources_for_critical_claim: int = Field(default=2, ge=1, le=5)
    minimum_full_text_sources_for_state_of_art_claim: int = Field(default=1, ge=1, le=5)
    minimum_support_confidence: float = Field(default=0.72, ge=0.0, le=1.0)
    allow_abstract_only_support: bool = False
    allow_preprint_as_sole_support: bool = False
    allow_search_snippet_as_evidence: bool = False
    require_exact_locator: bool = True
```

Presenter note — do not use the concept-note run for this beat

Run `ed462b3e` shows `Sources (0)` in Cockpit, and that is correct: its evidence base is bounded Deep Research dossiers, and it never invokes the acquire/verify chain. Tracing a claim there would mean narrating a pipeline the run did not execute. Use `1f2c3a4d`, which did.

Expect the follow-up: "so evidence acquisition doesn't work?" The honest answer is the strong one — **verification works and acquisition is the weak link**. The gates held under pressure and reported a specific, actionable gap per claim (`recommended_action: search_again`, `blocking: true`). Source resolution coverage is the next piece of engineering, and it is the limitation you raise yourself in beat 8.

7:05–8:20 Intended model, actual model, and cost

~200 words

ON SCREEN

The application header showing `$6.6495 this run` for `ed462b3e`, then the `cost_ledger` rows for that run — sorted by cost, with the `outline` row in view.

SAY

"Every model call made through the gateway is written to the cost ledger with its run ID and node ID. The record includes the intended model, actual model, input and output tokens, and calculated cost.

Intended and actual model are stored separately because a configured fallback may execute during a provider failure or rate limit. In this run, one did. The outline node requested Claude Sonnet four-five, and a different model actually ran. The workflow still completed and the deliverable was still produced — but that substitution is a recorded fact rather than an invisible one, which is the entire point of storing both fields.

Per-node telemetry turns cost into an engineering control. Discovery is forty-three percent of the spend on this run, and the five parallel drafting chunks are another forty-five percent, at almost identical input sizes — which is what a correct fan-out should look like. I can then use a smaller model for classification, batch compression, cache stable retrieval results, or reserve a stronger model for final synthesis.

It also supports debugging. If a node produces output but has no cost entry, I would check whether its call bypassed the context-bound gateway."

Node	Intended	Actual	Calls	In	Out	USD
deep_research	gpt-5.6-sol	gpt-5.6-sol	4	357,723	35,249	2.8461
draft_chunk_4	claude-sonnet-4-5	claude-sonnet-4-5	1	178,129	8,000	0.6544
draft_chunk_5	claude-sonnet-4-5	claude-sonnet-4-5	1	178,126	4,872	0.6075
draft_chunk_3	claude-sonnet-4-5	claude-sonnet-4-5	1	177,987	4,072	0.5950
draft_chunk_1	claude-sonnet-4-5	claude-sonnet-4-5	1	178,115	3,519	0.5871
draft_chunk_2	claude-sonnet-4-5	claude-sonnet-4-5	1	178,167	3,076	0.5806
outline	claude-sonnet-4-5	gpt-5.6-terra	1	142,614	3,523	0.3275
planner	claude-opus-5	claude-opus-5	1	36,228	3,595	0.2830
graphnormalizer_1	claude-sonnet-4-5	claude-sonnet-4-5	1	9,015	7,859	0.1683
Total — run ed462b3e			12	1,436,104	73,765	6.6495

Artefact 4 · the single ledger record behind the fallback claim `Mongo eurskem_ai.cost_ledger · run ed462b3e`

```
{
  "run_id": "ed462b3e-4b25-4e54-a0be-c8e46b516e05",
  "node_id": "outline",
  "intended_model": "claude-sonnet-4-5",
  "model": "gpt-5.6-terra",      <-- a fallback actually ran
  "input_tokens": 142614, "output_tokens": 3523,
  "cost_usd": 0.327504
}
```

Presenter note — say where this number comes from

The **run-level** total is in the application header. The **per-node** breakdown is not yet joined into the Cockpit node inspector, which states as much in the UI: "Token/cost breakdown isn't joined into this view yet — see the run's overall cost summary for now." So show the ledger query, and say the sentence: "the data is recorded per node and per call; the panel that joins it into the node inspector is not built yet." Claiming a UI that is not there is the one thing that would undermine a beat about telemetry honesty.

8:20–9:30 Limitation and concrete engineering change

~200 words

ON SCREEN

Back to the evidence record from beat 6, or a short roadmap slide titled Evidence-quality gate. Optionally the failed Cockpit node as a reminder that failures are localised rather than hidden.

SAY

"One limitation I would address next is that traceability is not the same as source authority.

The platform can show that a sentence is linked to an exact passage. However, that passage may come from a weak study, an outdated publication, or evidence that is unsuitable for the claim. Exact matching cannot make that judgement reliably.

My engineering change would be an evidence-quality gate before drafting. I would add structured fields for source type, date, peer-review status, study design, retraction or correction status, and domain-specific quality criteria. Deterministic rules would block clearly unsuitable evidence, while a domain expert would review high-impact or uncertain claims.

The truth graph would store both passage support and the quality decision, including who approved it and under which policy version. I would test it with a versioned evaluation set containing supported and unsupported claims, conflicting evidence, retracted sources, and passages that appear relevant but do not justify the conclusion."

The screenshot displays the Eurskem AI Workflow Studio interface. On the left is a dark sidebar with navigation options: Workspace (Workflow Studio, Evaluation Lab, Operator Console), System (Cloud Map, Audit Log, Settings), and a user profile for Ayush Khandelwal. The main panel is titled 'Studio' and includes tabs for Library, Builder, Run history, Pipelines, and Proposal review. A top bar shows a cost of '\$0.0000 this run'. Below the tabs, a 'Concept Note to 10-Page Methodol...' is visible, along with a status 'Failed' and a timestamp 'Started 9:29:27 AM - 4m 6s elapsed'. A table shows execution statistics: Running (0), Completed (2), Waiting (0), Paused (0), Failed (1), Skipped (11), and Cancelled (0). A search bar for nodes is present, showing 14 of 14 nodes. A list of nodes includes 'outline', 'draft_chunk_1' through 'draft_chunk_5', 'assemble', 'figures', 'render_docx', and 'graphnormalizer_1' (Node - Failed). The central canvas shows a workflow graph with a 'COMPLETED' node, a 'GRAPHNORMALIZER' node (labeled 'graphnormalizer_1' with a duration of 244.9s and a red 'X FAILED' status), and a 'PLANNER' node (labeled 'SCIENTIFIC planner'). A 'React Flow' diagram is visible at the bottom right. On the right side, a red error message states: 'Workflow failed: OpenAIGateway: empty structured response for GraphExtraction. refusal=None', with a button to 'Open Run History and retry'.

Shot 10 · failure stays attached to a node — `graphnormalizer_1` failed after 244.9 s; upstream work is preserved as COMPLETED and everything downstream is explicitly SKIPPED rather than silently missing. Use this if you want a second limitation: the provider-level cause is surfaced verbatim, which is honest but not yet self-explanatory to a non-engineer.

Two other limitations that answer well, if pressed

Live environment evidence. The repository documents an IONOS topology, readiness gates and rollback; this walkthrough demonstrates a local build. Do not present deployment tooling as a running production service.

Source-access coverage. Beat 6 showed the gates holding because acquisition returned nothing. Broadening canonical-URL resolution and licensed full-text access is the change that would turn a correct refusal into a correct citation.

9:30–10:00 Closing

~100 words

ON SCREEN

Either a prepared slide showing Library, Guided Run, Cockpit, Builder, Evidence and Cost together, or — stronger — the rendered deliverable, so the last thing on screen is the thing the platform produced.

SAY

"To summarise, Eurskem AI connects the business experience and engineering controls around one workflow definition and one run. Non-technical users work in Guided Run, operators inspect Cockpit, authors use typed Builder controls, and reviewers can trace evidence, model choices, cost, and human decisions.

The central design choice is visibility: the work, evidence, failures, decisions, and cost remain inspectable instead of being hidden inside one model response. Thank you for watching."

Untitled proposal - Part B
- Draft

HORIZON EUROPE

Part B

Untitled proposal

Version Draft

Date 2026-08-04

Confidential

Table of contents

- 1. [Methodology: Harmonised, Quality-Aware Mapping and Circular Routing of Mediterranean Pruning Residues](#)
- 2. [1 Methodology](#)
- 3. [1.1 Methodological purpose and intervention logic](#)
- 4. [1.2 State of the art, evidence gaps and project positioning](#)
- 5. [1.3 Research questions and testable hypotheses](#)
- 6. [1.4 Pilot-territory comparative design and unit of analysis](#)
- 7. [2 Methodology](#)
- 8. [2.1 Objective 1—Parcel-level supply mapping and availability estimation](#)
- 9. [2.2 Objective 2—Feedstock quality characterisation and predictive decision support](#)
- 10. [2.3 Objective 3—Cascading matchmaking and logistics optimisation](#)
- 11. [2.4 Objective 4—Traceability and carbon-removal-readiness architecture](#)
- 12. [2.5 Integrated digital platform and pilot operating protocol](#)
- 13. [3.1 Common data model and interoperability specifications](#)
- 14. [3.2 LPIS acquisition and sampling frame](#)
- 15. [4. Evaluation framework](#)
- 16. [4.1 Validation design and counterfactual baselines](#)
- 17. [4.2 Model-performance and uncertainty metrics](#)
- 18. [4.3 Pilot demonstration KPIs and statistical analysis](#)
- 19. [4.4 4.4 Environmental and economic comparative assessment](#)
- 20. [4.5 Biochar Carbon-removal Certification-readiness assessment](#)
- 21. [5.1 Risk-based implementation and uncertainty management](#)
- 22. [5.2 Ethics, legal compliance and responsible AI](#)
- 23. [5.3 Open science, FAIR data and exploitation-aware access](#)
- 24. [5.4 Feasibility, governance and adaptive management](#)

Methodology: Harmonised, Quality-Aware Mapping and Circular Routing of Mediterranean Pruning Residues

1 Methodology

1.1 Methodological purpose and intervention logic

The MATCH-BIO project is a multi-country, mixed-methods, two-season demonstration of a digital parcel-to-end-use framework for vine and olive pruning residues. It addresses the gap between theoretical biomass availability estimates—which aggregate regional statistics into static inventories—and the operational requirements of industrial biorefinery procurement: real-time, quality-aware, traceable feedstock flows at the scale of individual agricultural parcels and pruning batches. The analytical units are deliberately nested to capture the full supply chain: the **parcel** (or tree block) as the origin of biomass; the **pruning batch** as the unit of quality characterisation; the **collection route** as the logistical optimisation domain; the **receiving facility** as the demand node with quality thresholds; the **route allocation** as the decision outcome; and the **pilot territory** as the governance and validation scale.

The intervention logic follows a theory of change in which better evidence, actor coordination and constrained routing are hypothesised—not assumed—to improve mobilisation and sustainability outcomes. Current practice in Mediterranean viticulture and olive cultivation is characterised by open-field burning or low-value on-farm disposal of pruning residues, driven by fragmented land ownership, uncertain residue volumes, unknown feedstock quality, and the absence of coordinated logistics. The MATCH-BIO framework intervenes at three points: (i) **supply estimation**, replacing static coefficients with parcel-level remote-sensing models calibrated against local field data; (ii) **quality prediction and measurement**, integrating ex-ante bioclimatic indicators with portable NIR sensors and laboratory analysis to characterise moisture, ash, C:N ratio, lignin and—where economically justified—bioactive compounds (resveratrol in vine shoots, oleuropein and hydroxytyrosol in olive wood); and (iii) **cascading allocation**, routing high-quality fractions to biorefineries for phytochemical extraction or biochar production, intermediate fractions to pellet or compost facilities, and only residual material to energy recovery, thereby maximising economic return per tonne while minimising transport emissions and ensuring traceability from parcel to end-use.

The project will test this framework across **ten pilot territories** distributed in five Mediterranean countries (Spain, Italy, France, Portugal, Greece), deliberately selected to represent contrasting archetypes rather than statistical representativeness. The archetypes are: **high-density industrial-volume** sites (e.g. Castilla-La Mancha vineyards, Jaén olive groves) where large, contiguous parcels enable mechanised harvesting and economies of scale; **premium viticulture** sites (e.g. Bordeaux, Champagne) where strict appellation rules and high-value terroirs justify precision management and phytochemical valorisation; **fragmented/mountainous** sites (e.g. Douro Valley, Tuscan hills) where small, dispersed parcels and steep slopes challenge logistics but offer opportunities for cooperative aggregation and local biochar loops; and **island/peripheral** sites (e.g. Crete, Sicily) where isolation from mainland biorefineries tests the feasibility of decentralised, standalone circular-economy models. This typology ensures that the toolbox is validated under diverse agronomic, logistical and institutional conditions, and that transferability can be assessed

Shot 11 · what the run produced — a Horizon Europe Part B methodology section, roughly 11,900 words across a 24-entry contents, rendered to DOCX from the five assembled chunks. The renderer flags its own overrun (25 pages against a configured limit of 10) and marks the run **NEEDS INPUT** rather than truncating silently — a good last frame, because it is the whole thesis in one screen.

APPENDIX

Shot list, artefacts, and honesty notes

Shot list

Shot	File	Beat
1	01-workflow-library.png	1 — Introduction
2	02-library-concept-note-card.png	2 — Business goal
3	03-guided-run-overview.png	3 — Guided Run
4	06-cockpit-graph.png	3 — Cockpit
5	09-builder-canvas-palette.png	4 — Builder layout
6	12-builder-preflight.png	4 — Zero-token preflight
7	11-builder-configure.png	4 — Typed configuration
8	14-builder-map-data.png	4 — Data mapping
9	13-builder-versions.png	4 — Version history
10	07-cockpit-node-detail.png	8 — Failure localisation
11	23-output-proposal-cover-toc.png	9 — Closing

Optional B-roll from the same capture set: 10b-builder-graph-fanout.png (the five-way drafting fan-out, excellent under beat 4's "branch test" line), 17-run-detail-nodes.png (per-node durations, good under beat 7's cost narration), 20-cockpit-audit.png, 24-output-proposal-body.png.

Artefacts referenced in the script

Artefact	Where it lives	Beat
Node execution boundary	<code>app/runtime/compiler.py:63</code>	5
Atomic save + version record	<code>app/workflow/builder_store.py:34,135</code>	4
Fail-closed evidence policy	<code>app/evidence/models.py:46</code>	6
Acquisition report	<code>run 1f2c3a4d · acquire_research_sources.report</code>	6
QA report, 0 verified claims	<code>run 1f2c3a4d · verify_evidence.qa_report</code>	6
Fallback ledger record	<code>eurskem_ai.cost_ledger · ed462b3e / outline</code>	7

Recording and honesty notes

- Rehearse at roughly 125–135 words per minute and pause briefly when changing screens.
- Prepare the workflow, run, evidence record, cost query and code tabs before recording.
- Show a fallback because one genuinely exists in `run ed462b3e` — not as a hypothetical.
- Say "this is a local build" once, early. It costs two seconds and removes an entire class of doubt.
- Do not claim production adoption, measured accuracy, time savings, live deployment health, or regulatory approval.
- When a surface does not exist yet — per-node cost in the inspector — say so. The script is stronger for it.

Strong closing line, if you need one beyond the script

I built the reusable orchestration and evidence-control platform; proposal generation is one demanding use case that proves the abstraction. The design keeps provider automation bounded, sources traceable, people accountable, and failures recoverable — and when the evidence gates could not be satisfied, the system returned nineteen blocking issues instead of nineteen plausible citations.